

1. Finding Duplicates — $O(n^2)$ Time, $O(1)$ Space

Problem Statement

Find Duplicate in Array.

Given a read-only array of n integers between 1 and n , find one number that repeats.

Input Format:

First Line — Number of elements (n)

Second Line — n space-separated elements

Output Format:

Element x — That is repeated

C Code

```
/*
 * Problem 1: Finding Duplicates -  $O(n^2)$  Time,  $O(1)$  Space
 * Given a read-only array of  $n$  integers between 1 and  $n$ , find one number that repeats.
 * Brute force: compare every pair of elements.
 */
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int duplicate = -1;
    for (int i = 0; i < n && duplicate == -1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                duplicate = arr[i];
                break;
            }
        }
    }

    if (duplicate != -1)
        printf("%d\n", duplicate);
    else
        printf("No duplicate found\n");

    return 0;
}
```

Sample Input

```
5
1 1 2 3 4
```

Sample Output

```
1
```

2. Finding Duplicates — $O(n)$ Time, $O(1)$ Space

Problem Statement

Find Duplicate in Array.

Given a read-only array of n integers between 1 and n , find one number that repeats.

Input Format:

First Line — Number of elements (n)

Second Line — n space-separated elements

Output Format:

Element x — That is repeated

C Code

```
/*
 * Problem 2: Finding Duplicates -  $O(n)$  Time,  $O(1)$  Space
 * Given a read-only array of  $n$  integers between 1 and  $n$ , find one number that repeats.
 * Trick: use the array values as indices (Floyd's cycle detection / index marking works
 * only on mutable arrays, but since values lie in  $[1, n]$ , we can use the sign-marking
 * technique in  $O(n)$  time and  $O(1)$  extra space on a *copy* since the array is read-only
 * conceptually; here we simulate read-only by treating input as fixed and using
 * Floyd's Tortoise and Hare (cycle detection) approach, which needs no modification
 * of the array at all - true  $O(1)$  space,  $O(n)$  time, works on read-only arrays.
 */
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    /* Floyd's Cycle Detection on the "linked list" formed by arr[i] -> arr[arr[i]] */
    int slow = arr[0];
    int fast = arr[0];

    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    } while (slow != fast);

    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }

    printf("%d\n", slow);

    return 0;
}
```

Sample Input

1 1 2 3 4

Sample Output

1

3. Intersection of 2 Sorted Arrays — $O(m \cdot n)$ Time, $O(1)$ Space

Problem Statement

Find the intersection of two sorted arrays — i.e., given two sorted arrays, find all the elements which occur in both arrays.

Input Format:

First line: T, the number of test cases. For each test case:

Line 1: N1 followed by N1 integers of the first array

Line 2: N2 followed by N2 integers of the second array

Output Format:

The intersection of the arrays printed on a single line

C Code

```
/*
 * Problem 3: Intersection of 2 sorted arrays -  $O(m \cdot n)$  Time,  $O(1)$  Space
 * Given 2 sorted arrays, print all elements which occur in both arrays.
 * Brute force: for every element of array A, scan through array B.
 */
#include <stdio.h>

int main() {
    int t;
    scanf("%d", &t);

    while (t--) {
        int n1;
        scanf("%d", &n1);
        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        int n2;
        scanf("%d", &n2);
        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int first = 1;
        for (int i = 0; i < n1; i++) {
            for (int j = 0; j < n2; j++) {
                if (a[i] == b[j]) {
                    if (!first) printf(" ");
                    printf("%d", a[i]);
                    first = 0;
                    break;
                }
            }
        }
        printf("\n");
    }

    return 0;
}
```

Sample Input

```
1
3 10 17 57
6 2 7 10 15 57 246
```

Sample Output

```
10 57
```

4. Intersection of 2 Sorted Arrays — $O(m+n)$ Time, $O(1)$ Space

Problem Statement

Find the intersection of two sorted arrays — i.e., given two sorted arrays, find all the elements which occur in both arrays.

Input Format:

First line: T, the number of test cases. For each test case:

Line 1: N1 followed by N1 integers of the first array

Line 2: N2 followed by N2 integers of the second array

Output Format:

The intersection of the arrays printed on a single line

C Code

```
/*
 * Problem 4: Intersection of 2 sorted arrays -  $O(m+n)$  Time,  $O(1)$  Space
 * Given 2 sorted arrays, print all elements which occur in both arrays.
 * Two-pointer technique exploiting the fact both arrays are sorted.
 */
#include <stdio.h>

int main() {
    int t;
    scanf("%d", &t);

    while (t--) {
        int n1;
        scanf("%d", &n1);
        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        int n2;
        scanf("%d", &n2);
        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int i = 0, j = 0, first = 1;
        while (i < n1 && j < n2) {
            if (a[i] < b[j]) {
                i++;
            } else if (a[i] > b[j]) {
                j++;
            } else {
                if (!first) printf(" ");
                printf("%d", a[i]);
                first = 0;
                i++;
                j++;
            }
        }
        printf("\n");
    }

    return 0;
}
```

Sample Input

```
1
3 10 17 57
6 2 7 10 15 57 246
```

Sample Output

```
10 57
```

5. Pair with Difference — $O(n^2)$ Time, $O(1)$ Space

Problem Statement

Given an array A of sorted integers and another non-negative integer k , find if there exist two indices i and j such that $A[j] - A[i] = k$, with $i \neq j$.

Input Format:

First line: n — number of elements in the array

Next line: n space-separated elements

Next line: k — non-negative integer

Output Format:

1 — If such a pair exists

0 — If no such pair exists

C Code

```
/*
 * Problem 5: Pair with Difference -  $O(n^2)$  Time,  $O(1)$  Space
 * Given a sorted array  $A$  and a non-negative integer  $k$ , find if there exist
 * indices  $i, j$  ( $i \neq j$ ) such that  $A[j] - A[i] = k$ .
 * Brute force: check every pair.
 */
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int k;
    scanf("%d", &k);

    int found = 0;
    for (int i = 0; i < n && !found; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && arr[j] - arr[i] == k) {
                found = 1;
                break;
            }
        }
    }

    printf("%d\n", found);

    return 0;
}
```

Sample Input

```
3
1 3 5
4
```

Sample Output

6. Pair with Difference — $O(n)$ Time, $O(1)$ Space

Problem Statement

Given an array A of sorted integers and another non-negative integer k , find if there exist two indices i and j such that $A[j] - A[i] = k$, with $i \neq j$.

Input Format:

First line: n — number of elements in the array

Next line: n space-separated elements

Next line: k — non-negative integer

Output Format:

1 — If such a pair exists

0 — If no such pair exists

C Code

```
/*
 * Problem 6: Pair with Difference -  $O(n)$  Time,  $O(1)$  Space
 * Given a sorted array  $A$  and a non-negative integer  $k$ , find if there exist
 * indices  $i, j$  ( $i \neq j$ ) such that  $A[j] - A[i] = k$ .
 * Two-pointer technique exploiting the sorted order of the array.
 */
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int k;
    scanf("%d", &k);

    int i = 0, j = 1, found = 0;

    if (k == 0) {
        /* Special case: look for any two equal adjacent elements (duplicate) */
        for (int idx = 1; idx < n; idx++) {
            if (arr[idx] == arr[idx - 1]) {
                found = 1;
                break;
            }
        }
    } else {
        while (i < n && j < n) {
            if (i == j) {
                j++;
                continue;
            }
            if (j >= n) break;

            int diff = arr[j] - arr[i];
            if (diff == k) {
                found = 1;
                break;
            } else if (diff < k) {

```

```
        j++;
    } else {
        i++;
        if (i == j) j++;
    }
}

printf("%d\n", found);

return 0;
}
```

Sample Input

```
3
1 3 5
4
```

Sample Output

```
1
```